Package indexes

By default, uv uses the Python Package Index (PyPI) for dependency resolution and package installation. However, uv can be configured to use other package indexes, including private indexes, via the `[[tool.uv.index]]` configuration option (and `--index`, the analogous command-line option).

Defining an index

To include an additional index when resolving dependencies, add a `[[tool.uv.index]]` entry to your `pyproject.toml`:

```toml
[[tool.uv.index]]
# Optional name for the index.
name = "pytorch"
# Required URL for the index.
url = "https://download.pytorch.org/whl/cpu"
```

Indexes are prioritized in the order in which they're defined, such that the first index listed in the configuration file is the first index consulted when resolving dependencies, with indexes provided via the command line taking precedence over those in the configuration file.

By default, uv includes the Python Package Index (PyPI) as the "default" index, i.e., the index used when a package is not found on any other index. To exclude PyPI from the list of indexes, set `default = true` on another index entry (or use the `--default-index` command-line option):

```toml
[[tool.uv.index]]
name = "pytorch"
url = "https://download.pytorch.org/whl/cpu"
default = true
```

The default index is always treated as lowest priority, regardless of its position in the list of indexes.

Index names may only contain alphanumeric characters, dashes, underscores, and periods, and must be valid ASCII.

When providing an index on the command line (with `--index` or `--default-index`) or through an environment variable (`UV_INDEX` or `UV_DEFAULT_INDEX`), use its URL, a configured name, or the `<name>=<url>` syntax:

```
# On the command line.
$ uv lock --index pytorch=https://download.pytorch.org/whl/cpu
# Via an environment variable.
$ UV_INDEX=pytorch=https://download.pytorch.org/whl/cpu uv lock
```

With `--preview-features index-by-name`, configured index names take precedence over matching paths.

Pinning a package to an index

A package can be pinned to a specific index by specifying the index in its `tool.uv.sources` entry. For example, to ensure that `torch` is *always* installed from the `pytorch` index, add the following to your `pyproject.toml`:

```toml
[tool.uv.sources]
torch = { index = "pytorch" }

[[tool.uv.index]]
name = "pytorch"
url = "https://download.pytorch.org/whl/cpu"
```

Similarly, to pull from a different index based on the platform, you can provide a list of sources disambiguated by environment markers:

```toml title="pyproject.toml" [project] dependencies = ["torch"]

[tool.uv.sources] torch = [ { index = "pytorch-cpu", marker = "sys_platform == 'darwin'"}, { index = "pytorch-cu130", marker = "sys_platform != 'darwin'"},]

[[tool.uv.index]] name = "pytorch-cpu" url = "https://download.pytorch.org/whl/cpu"

[[tool.uv.index]] name = "pytorch-cu130" url = "https://download.pytorch.org/whl/cu130"


An index can be marked as `explicit = true` to prevent packages from being installed
from that index
unless explicitly pinned to it. For example, to ensure that `torch` is installed from
the `pytorch`
index, but all other packages are installed from PyPI, add the following to your
`pyproject.toml`:

```toml
[tool.uv.sources]
torch = { index = "pytorch" }

[[tool.uv.index]]
name = "pytorch"
url = "https://download.pytorch.org/whl/cpu"
explicit = true
```

Named indexes referenced via `tool.uv.sources` must be defined within the project's `pyproject.toml` file; indexes provided via the command-line, environment variables, or user-level configuration will not be recognized.

If an index is marked as both `default = true` and `explicit = true`, it will be treated as an explicit index (i.e., only usable via `tool.uv.sources`) while also removing PyPI as the default index.

Searching across multiple indexes

By default, uv will stop at the first index on which a given package is available, and limit resolutions to those present on that first index (`first-index`).

For example, if an internal index is specified via `[[tool.uv.index]]`, uv's behavior is such that if a package exists on that internal index, it will *always* be installed from that internal index, and never from PyPI. The intent is to prevent "dependency confusion" attacks, in which an attacker publishes a malicious package on PyPI with the same name as an internal package, thus causing the malicious package to be installed instead of the internal package. See, for example, the `torchtriton` attack from December 2022.

To opt in to alternate index behaviors, use the`--index-strategy` command-line option, or the `UV_INDEX_STRATEGY` environment variable, which supports the following values:

- `first-index` (default): Search for each package across all indexes, limiting the candidate versions to those present in the first index that contains the package.
- `unsafe-first-match`: Search for each package across all indexes, but prefer the first index with a compatible version, even if newer versions are available on other indexes.
- `unsafe-best-match`: Search for each package across all indexes, and select the best version from the combined set of candidate versions.
```

While `unsafe-best-match` is the closest to pip's behavior, it exposes users to the risk of "dependency confusion" attacks.

Authentication

Most private package indexes require authentication to access packages, typically via a username and password (or access token).

!!! tip

```
See the dedicated guides for authenticating with specific private index providers:
[Azure Artifacts](../guides/integration/azure.md),
[Google Artifact Registry](../guides/integration/google.md),
[AWS CodeArtifact](../guides/integration/aws.md), and
[JFrog Artifactory](../guides/integration/jfrog.md).
```

Providing credentials directly

Credentials can be provided directly via environment variables or by embedding them in the URL.

For example, given an index named `internal-proxy` that requires a username (`public`) and password (`koala`), define the index (without credentials) in your `pyproject.toml`:

```toml
[[tool.uv.index]]
name = "internal-proxy"
url = "https://example.com/simple"
```

From there, you can set the `UV_INDEX_INTERNAL_PROXY_USERNAME` and `UV_INDEX_INTERNAL_PROXY_PASSWORD` environment variables, where `INTERNAL_PROXY` is the uppercase version of the index name, with non-alphanumeric characters replaced by underscores:

```shell
export UV_INDEX_INTERNAL_PROXY_USERNAME=public
export UV_INDEX_INTERNAL_PROXY_PASSWORD=koala
```

By providing credentials via environment variables, you can avoid storing sensitive information in the plaintext `pyproject.toml` file.

Alternatively, credentials can be embedded directly in the index definition:

```toml
[[tool.uv.index]]
name = "internal"
url = "https://public:koala@pypi-proxy.corp.dev/simple"
```

For security purposes, credentials are *never* stored in the `uv.lock` file; as such, uv *must* have access to the authenticated URL at installation time.

Using credential providers

In addition to providing credentials directly, uv supports discovery of credentials from netrc and keyring. See the HTTP authentication documentation for details on setting up specific credential providers.

By default, uv will attempt an unauthenticated request before querying providers. If the request fails, uv will search for credentials. If credentials are found, an authenticated request will be attempted.

!!! note

```
If a username is set, uv will search for credentials before making an unauthenticated
request.
```

Some indexes (e.g., GitLab) will forward unauthenticated requests to a public index, like PyPI — which means that uv will not search for credentials. This behavior can be changed per-index, using the `authenticate` setting. For example, to always search for credentials:

```
toml hl_lines="4" [[tool.uv.index]] name = "example" url = "https://example.com/
simple" authenticate = "always"
```

When `authenticate` is set to `always`, uv will eagerly search for credentials and error if credentials cannot be found.

Ignoring error codes

When using the first-index strategy, uv will stop searching across indexes if an HTTP 401 Unauthorized or HTTP 403 Forbidden status code is encountered. The one exception is that uv will ignore 403s when searching the `pytorch` index (since this index returns a 403 when a package is not present).

By default, uv will also stop resolution if an HTTP error is encountered when fetching distribution metadata or an archive from an index. Ignoring that error marks the affected package version as unavailable, allowing the resolver to try another version.

To configure which error codes are ignored for an index, use the `ignore-error-codes` setting. For example, to ignore 403s (but not 401s) for a private index:

```
[[tool.uv.index]]
name = "private-index"
url = "https://private-index.com/simple"
authenticate = "always"
ignore-error-codes = [403]
```

uv will always continue searching across indexes when it encounters a `404 Not Found`. This cannot be overridden.

Disabling authentication

To prevent leaking credentials, authentication can be disabled for an index:

```
toml hl_lines="4" [[tool.uv.index]] name = "example" url = "https://example.com/
simple" authenticate = "never"
```

When `authenticate` is set to `never`, uv will never search for credentials for the given index and will error if credentials are provided directly.

Customizing cache control headers

By default, uv will respect the cache control headers provided by the index. For example, PyPI serves package metadata with a `max-age=600` header, thereby allowing uv to cache package metadata for 10 minutes; and wheels and source distributions with a `max-age=365000000, immutable` header, thereby allowing uv to cache artifacts indefinitely.

To override the cache control headers for an index, use the `cache-control` setting:

```
[[tool.uv.index]]
name = "example"
url = "https://example.com/simple"
cache-control = { api = "max-age=600", files = "max-age=365000000, immutable" }
```

The `cache-control` setting accepts an object with two optional keys:

- `api`: Controls caching for Simple API requests (package metadata).
- `files`: Controls caching for artifact downloads (wheels and source distributions).

The values for these keys are strings that follow the HTTP Cache-Control syntax. For example, to force uv to always revalidate package metadata, set `api = "no-cache"`:

```
[[tool.uv.index]]
name = "example"
url = "https://example.com/simple"
cache-control = { api = "no-cache" }
```

This setting is most commonly used to override the default cache control headers for private indexes that otherwise disable caching, often unintentionally. We typically recommend following PyPI's approach to caching headers, i.e., setting `api = "max-age=600"` and `files = "max-age=365000000, immutable"`.

Requiring a hash algorithm

When an index advertises multiple hashes for a distribution, uv selects a single hash to record in the lockfile. To require a specific algorithm for distributions resolved from an index, use the `hash-algorithm` setting:

```
[tool.uv]
preview-features = ["index-hash-algorithm"]

[[tool.uv.index]]
name = "private-index"
url = "https://private-index.com/simple"
hash-algorithm = "sha256"
```

If a locked distribution does not advertise the required algorithm, uv will fail instead of falling back to another hash algorithm.

Configuring `exclude-newer` for an index

If you're using `exclude-newer`, you can configure a different cutoff for a specific index:

```
[[tool.uv.index]]
name = "internal"
url = "https://internal.example.com/simple"
exclude-newer = "7 days"
```

Index-specific values only affect packages served from that index. Package-specific `exclude-newer-package` overrides still take precedence.

If an index does not provide `upload-time` metadata, you can disable the cutoff for that index entirely:

```
[[tool.uv.index]]
name = "internal"
url = "https://internal.example.com/simple"
exclude-newer = false
```

"Flat" indexes

By default, `[[tool.uv.index]]` entries are assumed to be PyPI-style registries that implement the PEP 503 Simple Repository API. However, uv also supports "flat" indexes, which are local directories or HTML pages that contain flat lists of wheels and source distributions. In pip, such indexes are specified using the `--find-links` option.

To define a flat index in your `pyproject.toml`, use the `format = "flat"` option:

```
[[tool.uv.index]]
name = "example"
```

```
url = "/path/to/directory"
format = "flat"
```

Flat indexes support the same feature set as Simple Repository API indexes (e.g., `explicit = true`); you can also pin a package to a flat index using `tool.uv.sources`.

`--index-url` and `--extra-index-url`

In addition to the `[[tool.uv.index]]` configuration option, uv supports pip-style `--index-url` and `--extra-index-url` command-line options for compatibility, where `--index-url` defines the default index and `--extra-index-url` defines additional indexes.

These options can be used in conjunction with the `[[tool.uv.index]]` configuration option, and follow the same prioritization rules:

- The default index is always treated as lowest priority, whether defined via the legacy `--index-url` argument, the recommended `--default-index` argument, or a `[[tool.uv.index]]` entry with `default = true`.
- Indexes are consulted in the order in which they're defined, either via the legacy `--extra-index-url` argument, the recommended `--index` argument, or `[[tool.uv.index]]` entries.

In effect, `--index-url` and `--extra-index-url` can be thought of as unnamed `[[tool.uv.index]]` entries, with `default = true` enabled for the former. In that context, `--index-url` maps to `--default-index`, and `--extra-index-url` maps to `--index`.